Table of contents

Prerequisites

Using ECP to enable and record ...

Enabling Perfetto

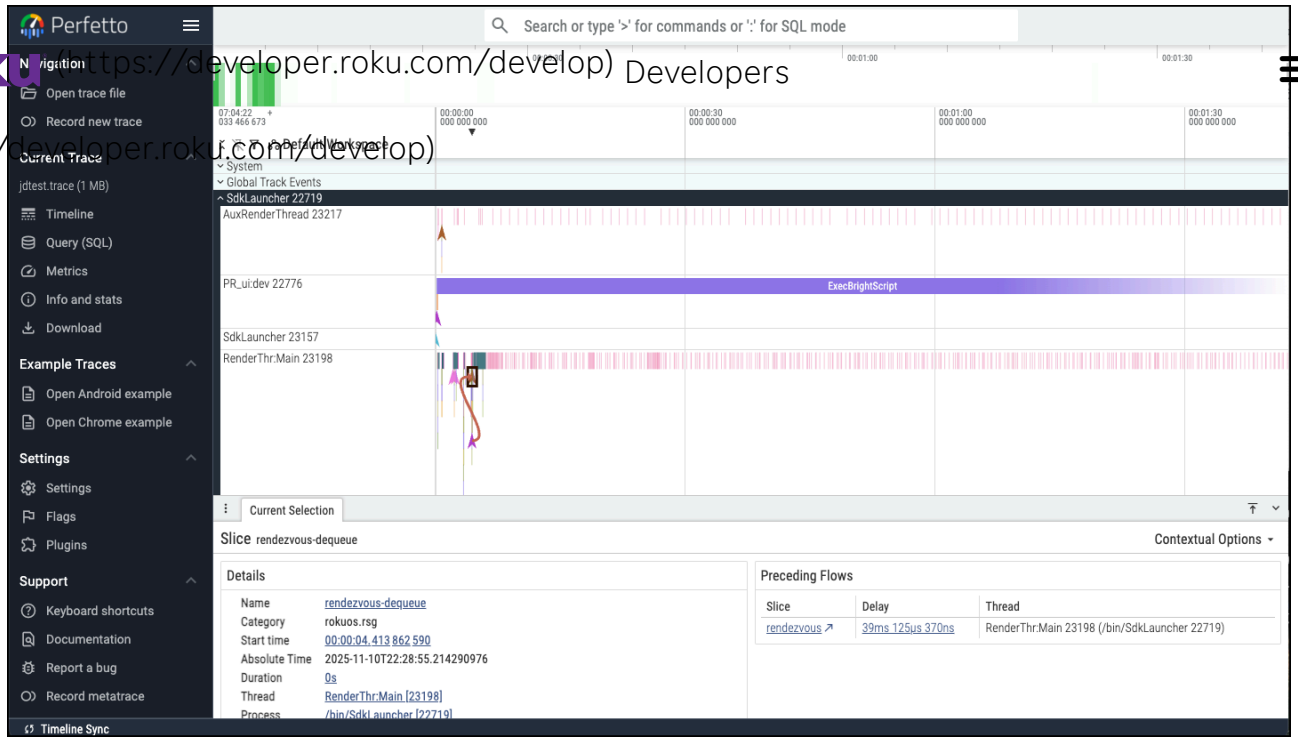
Recording a Perfetto trace

Show all...

Roku app tracing (with Perfetto)

Starting with Roku OS 15.1, you can use Perfetto (https://perfetto.dev/docs/) to record, analyze, and visualize traces of your Roku apps to pinpoint where you can reduce resource consumption and optimize performance. Tracing captures and visualizes the events in your app on a timeline, which provides you with a detailed graphical view of what your app is doing over time.

With Roku ECP and a Websocket client, you can launch your app, record and save a trace, and then open it in Perfetto. You can then explore the trace in Perfetto by using the WASD keys on your keyboard to zoom and pan, and your mouse to expand process tracks (rows) into their constituent thread tracks. You can also execute SQL-based queries in Perfetto.



Prerequisites

To record a trace, developers need the following:

- Roku OS 15.1 (or later)
- A Roku device with developer mode enabled (/docs/developer-program/getting-started/developer-setup.md).
- Roku app (you can record a trace with an app running in a sideloaded (/docs/developer-program/getting-started/developer-setup.md#sideloading-channels), beta, or production environment).
    - For a sideloaded app, the manifest must enable the run_as_process attribute (run_as_process=1).
    - For a beta (https://developer.roku.com/docs/developer-program/publishing/channel-publishing-guide.md#beta-channel-guidelines) or production (https://developer.roku.com/docs/developer-program/publishing/channel-publishing-guide.md#public-channel-guidelines) app, the developer must own the app (your device must be keyed with key used to sign the app package (/docs/developer-program/publishing/packaging-channels.md#rekeying-from-an-existing-package)). T
- Trace recording app. You can record an app trace via ECP and a websocket client.

Using ECP to enable and record Perfetto traces

You can enable and record a Perfetto trace with ECP following these steps:

Enabling Perfetto

To enable tracing for an app, send the enable Perfetto command with the channel ID:

```
curl -X POST "http://192.168.1.86:8060/perfetto/enable/dev"
```

The ECP response will have the following syntax:

```
<?xml version="1.0" encoding="UTF-8" ?>
<perfetto-enable>
  <enabled-channels>
    <channel>dev</channel>
  </enabled-channels>
  <timestamp>1762473265350</timestamp>
  <timestamp-end>1762473265350</timestamp-end>
  <status>OK</status>
</perfetto-enable>
```

Once enabled, tracing starts automatically each time the app is launched

> ECP does not have a corresponding disable Perfetto command. The list of enabled apps is cleared on reboot.

Recording a Perfetto trace

To record trace data for a session, using a websocket client to connect to the device (for example, websocat (https://github.com/vi/websocat)). The websocket emits a stream of bytes, which is the Protobuf-encoded Perfetto trace from the device.

```
websocat --binary ws://$ip:8060/perfetto-session > perfetto_data.trace
```

Data stops streaming when the client disconnects. If the client reconnects, the stream resumes.

> - Roku only supports a single websocket connection at a time.

Adding custom trace data

You can use the roPerfetto BrightScript component to capture custom events in a Perfetto trace. The types of events you can record include instantaneous, duration, scoped, and flow events.

Roku (https://developer.roku.com/develop) Developers
(https://developer.roku.com/develop)

Event	Description	Example	Snippet

Instantaneous	Events without a duration	keypress	`tracer = CreateObject("roPerfetto")`
			`tracer.instantEvent("my_instant_event")`
			`tracer.instantEvent("my_instant_event", {debu`

Duration	Events with a beginning and an end	long function	`sub myfunc()`
			`  tracer = CreateObject("roPerfetto")`
			`  params = {debug_1: 42, debug_2: "hello"}`
			`  tracer.beginEvent("my_duration_event", par`
			`  do_stuff()`
			`  tracer.endEvent()`
			`end sub`

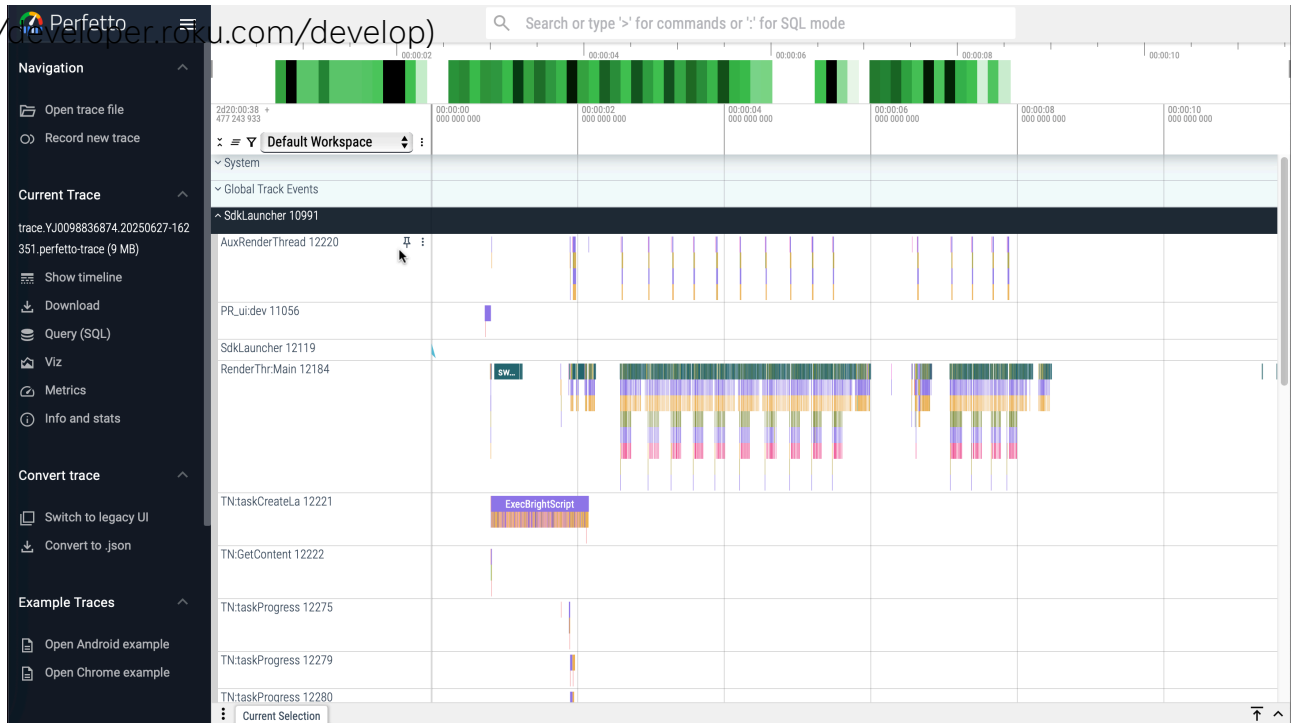
Scoped	Similar to duration events, but the end-event is generated automatically when the object returned from the call is released.		`sub myfunc()`
			`  tracer = CreateObject("roPerfetto")`
			`  params = {debug_1: 42, debug_2: "hello"}`
			`  scoped_event = tracer.createScopedEvent("`
			`  do_stuff()`
			`  ' end event auto-created when scoped_ever`
			`end sub`

Flow	Creation of a “flow” of events from one piece of code to another.	A Task thread to the Render thread.	`sub func1()`
			`  flowId = 42  ' A user-selected unique unsign`
			`  tracer = CreateObject("roPerfetto")`
			`  tracer.flowEvent(flowId, "my_flow_event_1")`
			`end sub`
			`sub func2()`
			`  flowId = 42`
			`  tracer = CreateObject("roPerfetto")`
			`  tracer.flowEvent(flowId, "my_flow_event_2")`
			`end sub`
			`sub func3()`
			`  flowId = 42`
			`  tracer = CreateObject("roPerfetto")`
			`  tracer.terminateFlow(flowId, "my_flow_even`
			`end sub`

Visualizing trace files in Perfetto

You can open trace files with the Perfetto UI (https://ui.perfetto.dev/) to anaylze them and pinpoint potential optimizations. Perfetto features a timeline view that provides a visual representation of the trace.

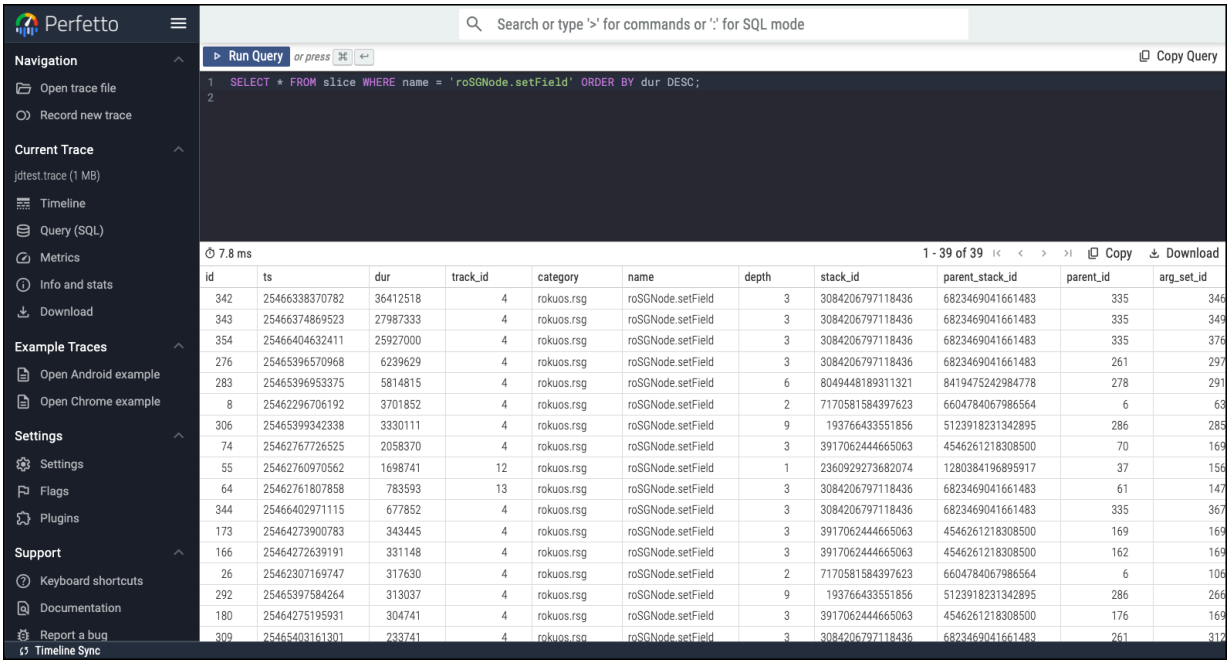
You can use the W,A,S,D keys on your keyboard to zoom and pan, and your mouse to expand process tracks (rows) into their constituent thread tracks.



Using PerfettoSQL to query traces

You can query the data in a trace using PerfettoSQL (https://perfetto.dev/docs/analysis/perfetto-sql-getting-started) following these steps:

1. Click Query (SQL) in the left sidebar in the Perfetto UI.
2. Enter your query in the editor.
3. Click Run Query.



id	ts	dur	track_id	category	name	depth	stack_id	parent_stack_id	parent_id	arg_set_id
342	25466338370782	36412518	4	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	335	346
343	25466374869523	27987333	4	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	335	349
354	25466404632411	25927000	4	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	335	376
276	25465396570968	6239629	4	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	261	297
283	25465396953375	5814815	4	rokuos.rsg	roSGNode.setField	6	8049448189311321	8419475242984778	278	291
8	25462296706192	3701852	4	rokuos.rsg	roSGNode.setField	2	7170581584397623	6604784067986564	6	63
306	25465399342338	3330111	4	rokuos.rsg	roSGNode.setField	9	193766433551856	5123918231342895	286	285
74	25462767726525	2058370	4	rokuos.rsg	roSGNode.setField	3	3917062444665063	4546261218308500	70	169
55	25462760970562	1698741	12	rokuos.rsg	roSGNode.setField	1	2360929273682074	1280384196895917	37	156
64	25462761807858	783593	13	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	61	147
344	25466402971115	677852	4	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	335	367
173	25464273900783	343445	4	rokuos.rsg	roSGNode.setField	3	3917062444665063	4546261218308500	169	169
166	25464272639191	331148	4	rokuos.rsg	roSGNode.setField	3	3917062444665063	4546261218308500	162	169
26	25462307169747	317630	4	rokuos.rsg	roSGNode.setField	2	7170581584397623	6604784067986564	6	106
292	25465397584264	313037	4	rokuos.rsg	roSGNode.setField	9	193766433551856	5123918231342895	286	266
180	25464275195931	304741	4	rokuos.rsg	roSGNode.setField	3	3917062444665063	4546261218308500	176	169
309	25465403161301	233741	4	rokuos.rsg	roSGNode.setField	3	3084206797118436	6823469041661483	261	312

The following examples demonstrate some of the use cases for querying your trace data:

Use case	Query
Find all rendezvous in order of duration (long rendezvous often cause dropped frames and may indicate inefficient observer functions).	`SELECT * FROM slices WHERE name = 'rendezvous' ORDER BY dur DESC;`
Find long executions of swapBuffers which indicate places where the channel may be dropping frames	`SELECT * FROM slice WHERE name = 'swapBuffers' ORDER BY dur DESC;`
Find places where the channel is handling a key press with `OnKeyEvent()`	`SELECT * FROM slice WHERE name = 'keyEvent' ORDER BY dur DESC;`
Find all of the observers being called in the app.	`SELECT * FROM slice WHERE name = 'observer.callback' ORDER BY dur D`
Find all of the places where the channel is calling `setField`.	`SELECT * FROM slice WHERE name = 'roSGNode.setField' ORDER BY dur`

Glossary

Roku's Perfetto-based app tracing solution exposes a number of terms that Roku developers may be unfamiliar with:

Term	**Defintion**
SdkLauncher	A Roku OS plugin that provides an environment for running SDK apps in a sandboxed process.
PR_ui	The main BrightScript thread.

Term	Defintion
RenderThread	The primary SceneGraph render thread.
AuxRenderThread	A Roku OS thread used to offload some rendering tasks from the main UI thread. This helps improve the responsiveness and smoothness of the user interface.
TN:[function name]	A Task thread spawned by a Task node (/docs/references/scenegraph/control-nodes/task.md) . The function name represents the name of the Task function defined by the app.
ExecBrightScript	A slice representing the BrightScript engine processing app code.
render	A slice representing the process of drawing or composing a frame for display.
swapBuffers	A slice representing the operation of presenting a newly rendered frame to the display.
consumeAllTasks	A slice representing the render thread as it is processing messages from the Render Thread Queue that are waiting. These can include rendezvous as well as messages sent using the roRenderThreadQueue component (/docs/references/brightscript/components/rorenderthreadqueue.md).
bscCopyToDomainEx	A slice representing data being copied. For example, when getting or setting a field on a Task node (/docs/references/scenegraph/control-nodes/task.md) from the Render thread.

Previous Next

Terms and Conditions

B2B Privacy Notice (https://docs.roku.com/published/b2bprivacypolicy/en/us)

Developer Site (https://developer.roku.com)

About ads & cookies (https://docs.roku.com/published/cookiepolicy)

My Privacy Choices
(https://privacy.roku.com/ccpa)
Developer Terms & Agreements (https://developer.roku.com/docs/features/legal/developer-terms.md)

CA B2B Privacy Notice

© 2008-2026 Roku, Inc. (https://www.roku.com)